

1. DOCUMENTATION

1.1 Importing Libraries

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.impute import KNNImputer
from sklearn.preprocessing import LabelEncoder
```

Purpose: Load the Python toolkits needed for data handling (pandas, numpy), splitting data into train/test sets, filling missing values (KNNImputer), and converting text categories into numbers (LabelEncoder).

Interpretation: Before any analysis can begin, the project loads its toolbox. Think of this as laying out the instruments before a procedure — nothing is being analyzed yet, we're just making sure every tool needed later is ready to use.

pandas and NumPy are imported for handling datasets, performing numerical computations, and manipulating data efficiently. Scikit-learn libraries such as train_test_split, KNNImputer, and LabelEncoder are imported to split the dataset into training and testing sets, handle missing values intelligently, and convert categorical variables into numerical format. Loading these libraries prepares the programming environment and provides all the necessary tools required for data preprocessing, analysis, visualization, and model building.

1.2 Loading the Dataset

```
df = pd.read_csv("Risk_Alert_Classifier_Dataset_4600...csv")
```

Purpose: Read the raw customer dataset (4,600 records) into a pandas DataFrame so it can be explored and processed.

Interpretation: This is the moment the customer data — ages, incomes, credit scores, payment behavior, and more — is brought into the workspace. Everything from this point forward works from this single table of records.

The dataset is loaded into a Pandas DataFrame using the read_csv() function. This step imports all customer financial information, including demographic details, credit history, spending behavior, and the target variable indicating financial risk. Once loaded, the dataset becomes available for further processing, cleaning, visualization, and machine learning analysis. This serves as the foundation of the entire project because every subsequent operation is performed on this dataset.

2. First Look at the Data

2.1–2.2 Previewing Records

```
df.head()
df.tail()
```

Purpose: Display the first and last five rows to sanity-check that the file loaded correctly and to see what the columns look like.

Interpretation: A quick visual spot-check — like flipping to the front and back pages of a report to make sure nothing looks broken before reading it in full.

2.3 Dataset Size

```
df.shape
```

Purpose: Report the number of rows (customers) and columns (features) in the dataset.

Interpretation: This tells us how much data we actually have to work with — the size of the customer pool and how many pieces of information were recorded about each one.

The shape function returns the total number of rows and columns in the dataset. This information helps understand the overall size of the data and indicates how many customer records are available for training the machine learning model.

2.4 Column Data Types & Structure

```
df.info()
```

Purpose: List every column along with its data type (numeric, text, etc.) and how many non-missing values it has.

Interpretation: This is the 'table of contents' for the dataset — confirming which columns are numbers (like income), which are categories (like region), and flagging where data might be missing.

The info() function provides a complete overview of the dataset structure. It displays each column's data type, the number of non-null values, and the memory usage of the dataset. This information is essential because machine learning algorithms require the correct data types, and missing values must be handled before model training. This step also helps distinguish between numerical and categorical variables for appropriate preprocessing.

2.5 Counting Missing Values

```
df.isnull().sum()
```

Purpose: Count how many blank/missing entries exist in each column.

Interpretation: Missing data is common in real-world customer records — someone might not have reported their income, for example. This step locates exactly where those gaps are before deciding how to handle them.

3. Handling Missing Values

3.1 Filling Numerical Gaps with the Median

```
numerical_columns = ["age", "annual_income_inr", "credit_score",  
                     "credit_utilization_ratio", "monthly_spend_inr"]  
for column in numerical_columns:  
    df[column].fillna(df[column].median())
```

Purpose: Replace missing values in numeric fields (age, income, credit score, credit utilization, monthly spend) with the median of that column.

Interpretation: Rather than deleting incomplete records, the missing values are filled in with a typical value. The median is used instead of the average because it isn't thrown off by a few unusually high or low customers, giving a more realistic stand-in value.

3.2 Filling Categorical Gaps with the Mode

```
categorical_columns = ["region", "employment_type"]
for column in categorical_columns:
    df[column].fillna(df[column].mode()[0])
```

Purpose: Replace missing values in text categories (region, employment type) with the most frequently occurring category.

Interpretation: For non-numeric fields, there's no 'average' to fall back on, so the most common answer is used instead — the same logic as assuming a customer probably falls into the most typical group when their information wasn't recorded.

Categorical features such as region and employment type cannot be filled using mathematical averages. Therefore, missing values are replaced with the mode, which represents the most frequently occurring category in each column. This ensures that all records remain complete while maintaining consistency within categorical variables.

3.3 Confirming the Fix

```
df.isnull().sum()
df["region"] = df["region"].fillna(df["region"].mode()[0])
df["employment_type"] = df["employment_type"].fillna(df["employment_type"].mode()[0])
```

Purpose: Re-check for missing values and explicitly re-apply the fill to region and employment_type to guarantee they are fully populated.

Interpretation: A follow-up check to make sure the earlier fix actually stuck. This kind of double-checking is normal practice — it's the equivalent of proofreading a form after filling it in. After imputing missing values, the dataset is checked again to ensure that every missing value has been successfully handled. This verification step is important because most machine learning algorithms cannot process datasets containing null values. Confirming the absence of missing data ensures the dataset is ready for further preprocessing and model development.

4. Understanding the Data's Shape

4.1 Statistical Summary

```
df.describe()
```

Purpose: Generate summary statistics — count, mean, standard deviation, minimum, quartiles, and maximum — for every numeric column.

Interpretation: This gives a bird's-eye view of the customer base: what a typical income or credit score looks like, and how wide the range is between the lowest and highest values.

4.2 Data Types & Duplicate Check

```
df.dtypes
df.duplicated().sum()
```

Purpose: Confirm the data type of each column and count any fully duplicated rows.

Interpretation: Duplicate rows would mean the same customer record was counted twice, which could bias the model. This step confirms the dataset is clean of exact repeats.

5. Exploratory Data Analysis (Visual Checks)

5.1 Histograms of Numeric Features

```
df[numerical_columns].hist(figsize=(18,12), bins=20)
```

Purpose: Plot the distribution (spread and frequency) of every numeric column.

Interpretation: Histograms show whether a feature like income is evenly spread, clustered around a typical value, or skewed toward one end — useful context before feeding it into a model.

Histograms display the frequency distribution of numerical variables such as income, credit score, and monthly spending. These visualizations help identify whether variables are normally distributed, positively skewed, negatively skewed, or contain multiple peaks. Understanding feature distributions helps determine whether transformations or scaling techniques may be necessary before model training.

5.2 Box Plots for Outlier Detection

```
for column in numerical_columns:  
    plt.boxplot(df[column])
```

Purpose: Draw a box plot for each numeric feature to visualize its median, quartile range, and any extreme values.

Interpretation: Box plots make unusually high or low values (outliers) easy to spot at a glance — for example, a handful of customers with an extremely high number of failed logins compared to everyone else.

5.3 Correlation Heatmap

```
sns.heatmap(df[numerical_columns].corr(), annot=True, cmap="coolwarm")
```

Purpose: Calculate and visualize how strongly each numeric feature relates to every other numeric feature.

Interpretation: This reveals which factors tend to move together — for instance, whether higher debt balances tend to go hand-in-hand with higher credit utilization — which helps guide feature selection later.

5.4 Counting Outliers with the IQR Method

```
Q1 = df[column].quantile(0.25)  
Q3 = df[column].quantile(0.75)  
IQR = Q3 - Q1  
lower = Q1 - 1.5*IQR; upper = Q3 + 1.5*IQR  
outliers = df[(df[column] < lower) | (df[column] > upper)]
```

Purpose: Use the Interquartile Range (IQR) rule — a standard statistical method — to count how many values in each numeric column fall unusually far outside the normal range.

Interpretation: This step quantifies what the box plots showed visually: exactly how many customers in each feature look like statistical outliers, so a decision can be made about whether to remove them.

The Interquartile Range (IQR) method identifies observations that fall significantly outside the normal distribution of the data. Values below the lower boundary or above the upper boundary are considered statistical outliers. This objective statistical approach ensures that only genuinely extreme observations are identified for possible removal.

6. Cleaning Outliers

6.1 Removing Extreme Values

```

selected_columns = ["age", "avg_late_payment_days", "monthly_transaction_count",
                    "cash_advance_count_6m", "complaints_last_6m", "failed_login_attempts_3m",
                    "account_tenure_months"]
df_clean = df.copy()
for column in selected_columns:
    ...remove rows outside [Q1-1.5*IQR, Q3+1.5*IQR]...
print("Original Shape :", df.shape)
print("After Removing :", df_clean.shape)

```

Purpose: Create a cleaned copy of the dataset (df_clean) with extreme outlier rows removed from seven behavior-related columns.

Interpretation: A small number of customers with implausible or extreme values (for example, an unrealistic number of failed logins) are set aside so they don't distort the patterns the model later learns from the majority of normal customers.

7. Exploring the Target Variable and Categories

7.1 Risk Status Counts

```
df_clean["risk_status"].value_counts()
```

Purpose: Count how many customers fall into each risk category (e.g. Low Risk vs. High Risk).

Interpretation: This is the first direct look at the outcome the model is trying to predict, and how the two groups compare in size.

7.2 Credit Score by Risk Status

```
sns.boxplot(data=df_clean, x="risk_status", y="credit_score")
```

Purpose: Compare the spread of credit scores between the low-risk and high-risk groups.

Interpretation: As expected, this kind of chart typically shows that customers labeled 'high risk' cluster around lower credit scores — a sanity check that the labels in the data make real-world sense.

7.3 Class Balance Check

```
sns.countplot(data=df_clean, x="risk_status")
```

Purpose: Visualize the total count of each risk category side by side.

Interpretation: This confirms whether the dataset is balanced or lopsided — for example, if there are far more low-risk customers than high-risk ones, that imbalance needs to be addressed before training, or the model may learn to just guess "low risk" every time.

7.4 Categorical Features vs. Risk

```

for column in ["gender", "region", "employment_type"]:
    sns.countplot(data=df_clean, x=column, hue="risk_status")

```

Purpose: Break down risk status by gender, region, and employment type.

Interpretation: This checks whether certain groups (like a particular region or employment type) show up more often in the high-risk category, hinting at which categorical factors might matter to the model.

7.5 Pairwise Relationships & Distributions

```

sns.pairplot(df_clean[important_features], hue="risk_status")
sns.kdeplot(data=df_clean, x=column, fill=True) # for each numeric column

```

```
df_clean[column].value_counts().plot(kind="pie", autopct="%1.1f%%") # for each categorical column
```

Purpose: Examine how key financial features relate to one another and to risk status, using pair plots, density plots, and pie charts.

Interpretation: These visuals build intuition for which financial signals — income, credit score, debt, spending — most clearly separate low-risk from high-risk customers, and how the categorical groups are proportioned.

7.6 Feature Relationships with Credit Utilization

7.7 Spending Trend by Account Tenure

```
df_clean.groupby("account_tenure_months")["monthly_spend_inr"].mean().plot()
```

Purpose: Track the average monthly spend as account tenure (how long someone has held their account) increases.

Interpretation: This looks for a lifecycle pattern — do customers tend to spend more or less the longer they've held their account? Useful context for understanding customer behavior over time.

8. Preparing Features and Target for Modeling

8.1 Re-checking Missing Values

```
sns.heatmap(df.isnull(), cbar=False, cmap="viridis")  
# Median/mode fill re-applied to numerical and categorical columns  
print("Total Missing Values:", df.isnull().sum().sum())
```

Purpose: Visualize and confirm, one final time, that no missing values remain anywhere in the dataset before modeling begins.

Interpretation: This is the last quality gate before the data touches a machine learning algorithm — most algorithms simply cannot run if any values are missing, so this step guarantees a clean hand-off.

8.2 Splitting Features (X) from the Target (y)

```
X = df.drop("risk_status", axis=1)  
y = df["risk_status"]
```

Purpose: Separate the dataset into the input features (X) the model will learn from, and the outcome (y) it will try to predict.

Interpretation: This is the fundamental split at the heart of any supervised learning task: everything we know about a customer (X) versus the one thing we're trying to predict about them (y — their risk status).

8.3 Removing the Identifier Column

```
X = X.drop("customer_id", axis=1)
```

Purpose: Drop the customer_id column from the feature set.

Interpretation: A customer's ID number is just a label — it carries no real information about their risk. Leaving it in could confuse the model into learning meaningless patterns, so it's removed.

8.4 Separating Numeric and Categorical Features

```
numerical_features = X.select_dtypes(include=np.number).columns.tolist()  
categorical_features = X.select_dtypes(include="object").columns.tolist()
```

Purpose: Identify which remaining columns are numeric and which are text-based categories.

Interpretation: Numeric and categorical data need different treatment before a model can use them, so this step sorts the features into the right buckets.

8.5 Encoding Categorical Features

```
label_encoders = {}  
for column in categorical_features:  
    encoder = LabelEncoder()  
    X[column] = encoder.fit_transform(X[column].astype(str))
```

Purpose: Convert text categories (like region or employment type) into numeric codes using Label Encoding, so the model can process them mathematically.

Interpretation: Machine learning models work with numbers, not words. This step translates categories like "Salaried" or "Self-Employed" into numeric codes behind the scenes, while keeping the original meaning tracked in the encoder.

9. Splitting into Training and Testing Sets

9.1 Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.20, random_state=42, stratify=y)
```

Purpose: Divide the data into an 80% training portion (used to teach the model) and a 20% testing portion (held back to evaluate it fairly), keeping the same proportion of risk classes in both.

Interpretation: This is like studying from 80% of a textbook and then taking a test on the remaining, unseen 20% — it's the standard way to check whether a model has actually learned useful patterns rather than just memorized the data.

9.2 Verifying the Split

```
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)  
print(y.value_counts(normalize=True))  
print(y_train.value_counts(normalize=True))  
print(y_test.value_counts(normalize=True))
```

Purpose: Confirm the size of each split and that the ratio of low-risk to high-risk customers is preserved across the original, training, and testing sets.

Interpretation: This double-checks that the training and test sets are fair, representative samples of the whole customer base — not accidentally skewed toward one risk group.

9.3 Missing Value Audit on Splits

```
print(X_train.isnull().sum())  
print(X_test.isnull().sum())  
missing = pd.DataFrame({"Missing Values": ..., "Percentage": ...})
```

Purpose: Re-verify that neither the training nor testing feature sets contain missing values, and summarize this in a small table.

Interpretation: One more safety check specifically on the split data, since a bug earlier in the pipeline could otherwise slip missing values back in unnoticed.

10. K-Nearest Neighbors (KNN) Imputation

10.1 Building and Applying the KNN Imputer

```
imputer = KNNImputer(n_neighbors=5, weights="uniform")
X_train_imputed = imputer.fit_transform(X_train_copy)
X_test_imputed = imputer.transform(X_test)
```

Purpose: Fill any remaining missing values by looking at the 5 most similar customers (nearest neighbors) and using their values as a smarter estimate. The imputer is fit only on training data, then applied to both sets to avoid data leakage.

Interpretation: Instead of a single blanket average, this method fills gaps using customers who genuinely resemble the one with missing data — a more nuanced approach. Fitting only on the training data ensures the test set stays a fair, unseen evaluation, with no information 'leaking' backward from it.

11. Baseline Model: Logistic Regression

11.1 Training the Model

```
from sklearn.linear_model import LogisticRegression
logistic_model = LogisticRegression(random_state=42, max_iter=1000)
logistic_model.fit(X_train, y_train)
```

Purpose: Train a Logistic Regression model — a well-established, easily interpretable classification algorithm — on the training data to learn the relationship between customer features and risk status.

Interpretation: This is the project's first working model, and a sensible starting point. As explained in the theory section, Logistic Regression estimates the probability that a customer is high risk, which is then converted into a Low Risk / High Risk label using a threshold.

11.2 Generating Predictions

```
y_pred = logistic_model.predict(X_test)
```

Purpose: Use the trained model to predict risk status for the customers in the held-out test set.

Interpretation: These are the model's 'guesses' on customers it has never seen before — the true test of whether it generalizes well.

11.3 Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ...)
```

Purpose: Build and visualize a confusion matrix comparing predicted labels to actual labels.

Interpretation: The confusion matrix is the clearest single picture of model performance: how many customers were correctly classified, and — just as importantly — how many were mixed up in each direction.

11.4 Classification Report & Core Metrics

```
print(classification_report(y_test, y_pred))
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
```

Purpose: Calculate the model's overall Accuracy, Precision, Recall, and F1-Score on the test data.

Interpretation: These four numbers summarize how trustworthy the baseline model is. As the theory notes explain, accuracy alone can be misleading on imbalanced data, so precision, recall, and F1-score are tracked alongside it to get the full picture — especially how well the model catches genuinely high-risk customers.

12. Error Analysis: Type-I and Type-II Errors

12.1 Extracting TN, FP, FN, TP

```
TN, FP, FN, TP = confusion_matrix(y_test, y_pred).ravel()
```

Purpose: Break the confusion matrix down into its four individual counts: True Negatives, False Positives, False Negatives, and True Positives.

Interpretation: These four building blocks are the raw ingredients behind every other error metric calculated in this section.

12.2 Type-I and Type-II Errors

```
type_1_error = FP    # False Positive  
type_2_error = FN    # False Negative
```

Purpose: Report the count of Type-I Errors (a genuinely low-risk customer wrongly flagged as high-risk) and Type-II Errors (a genuinely high-risk customer wrongly passed as low-risk).

Interpretation: This is the most business-critical part of the evaluation. A Type-I Error means a good customer might get unfairly rejected or flagged. A Type-II Error means a risky customer slips through undetected — which, as the theory notes point out, is typically the more costly mistake in financial risk settings because it can lead directly to loan defaults.

12.3 False Positive Rate & True Positive Rate

```
false_positive_rate = FP / (FP + TN)  
true_positive_rate = TP / (TP + FN)
```

Purpose: Convert the raw error counts into rates: what proportion of actually-safe customers were wrongly flagged (FPR), and what proportion of actually-risky customers were correctly caught (TPR).

Interpretation: Turning raw counts into rates makes performance comparable regardless of how many customers are in each group — a cleaner way to judge, for instance, 'out of every 100 genuinely risky customers, how many did the model actually catch?'

12.4 Error Summary Table & Chart

```
error_analysis = pd.DataFrame({...})  
plt.bar(["Type-I Error", "Type-II Error"], [FP, FN])
```

Purpose: Consolidate TN, FP, FN, TP, FPR, and TPR into a single reference table, and chart the two error types side by side.

Interpretation: A one-glance summary for stakeholders: which type of mistake is the model making more often, and by how much — directly informing whether the model's current balance of caution is acceptable for the business.

12.5 Risk Class Distribution Review

```
class_distribution = y.value_counts()
sns.countplot(x=y)
plt.pie(class_distribution, labels=["Low Risk", "High Risk"], autopct="%1.1f%%")
```

Purpose: Revisit how imbalanced the full dataset is between low-risk and high-risk customers.

Interpretation: This sets up the next major step: because one class is much rarer than the other, the model may be under-trained on high-risk examples — which is exactly what causes the kinds of errors just analyzed.

13. Balancing the Data with SMOTE

13.1 Before SMOTE

```
from imblearn.over_sampling import SMOTE
print(y_train.value_counts())
sns.countplot(x=y_train)
```

Purpose: Check and visualize the class balance of the training set before any correction is applied.

Interpretation: Confirms the imbalance problem directly in the training data — the exact data the model learns from — rather than just the overall dataset.

13.2 Applying SMOTE

```
smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
```

Purpose: Generate synthetic examples of the minority class (high-risk customers) so the training set contains a roughly equal number of both classes.

Interpretation: SMOTE is a common way to explain to non-technical stakeholders: rather than just copying existing high-risk customer records, it creates new, realistic 'blended' examples based on real high-risk customers' characteristics — giving the model more balanced material to learn from.

13.3 After SMOTE

```
print(y_train_smote.value_counts())
sns.countplot(x=y_train_smote)
```

Purpose: Confirm that the training set is now balanced between the two risk classes.

Interpretation: A visual and numeric confirmation that the fix worked — the training data going into the next model is no longer lopsided.

14. Logistic Regression Retrained on Balanced Data

14.1 Training on SMOTE Data

```
balanced_model = LogisticRegression(random_state=42, max_iter=1000)
balanced_model.fit(X_train_smote, y_train_smote)
y_pred_smote = balanced_model.predict(X_test)
```

Purpose: Retrain a fresh Logistic Regression model using the SMOTE-balanced training data, then predict on the same untouched test set as before.

Interpretation: This is a like-for-like rerun of the baseline model, with the only change being a more balanced training diet — isolating exactly how much the imbalance was hurting performance.

14.2 Evaluating the Balanced Model

```
print(classification_report(y_test, y_pred_smote))
cm_smote = confusion_matrix(y_test, y_pred_smote)
```

Purpose: Generate a new classification report and confusion matrix for the SMOTE-trained model.

Interpretation: The same yardstick used on the baseline model is applied here, keeping the comparison fair and apples-to-apples.

14.3 Baseline vs. SMOTE Comparison

```
comparison = pd.DataFrame({"Metric": [...], "Baseline": [...], "After SMOTE": [...]}
comparison.set_index("Metric").plot(kind="bar")
```

Purpose: Place the baseline and SMOTE-trained model's Accuracy, Precision, Recall, and F1-Score side by side in a table and bar chart.

Interpretation: This is the direct answer to "did balancing the data actually help?" — typically showing an improved ability to catch high-risk customers (recall), sometimes at a small cost to precision, which is a trade-off worth discussing with the business.

15. Trying Tree-Based Models

15.1 Decision Tree

```
from sklearn.tree import DecisionTreeClassifier
decision_tree = DecisionTreeClassifier(random_state=42)
decision_tree.fit(X_train_smote, y_train_smote)
y_pred_dt = decision_tree.predict(X_test)
```

Purpose: Train a Decision Tree — a model that learns a series of yes/no rules — on the balanced training data and predict on the test set.

Interpretation: A Decision Tree offers a different, rule-based way of thinking about risk (e.g. "if credit score is below X and late payments are above Y, flag as high risk"), which is often easier to explain than the probability-based logic of Logistic Regression.

15.2 Random Forest

```
from sklearn.ensemble import RandomForestClassifier
random_forest = RandomForestClassifier(n_estimators=100, random_state=42)
random_forest.fit(X_train_smote, y_train_smote)
y_pred_rf = random_forest.predict(X_test)
```

Purpose: Train a Random Forest — an ensemble of 100 individual decision trees whose votes are combined — on the balanced training data.

Interpretation: Random Forest is generally more robust than a single Decision Tree because it averages the opinions of many slightly different trees, which tends to reduce overfitting and produce steadier, more reliable predictions.

15.3 Evaluating Both Models

```
print(classification_report(y_test, y_pred_dt))
print(classification_report(y_test, y_pred_rf))
cm_dt = confusion_matrix(y_test, y_pred_dt)
cm_rf = confusion_matrix(y_test, y_pred_rf)
```

Purpose: Generate classification reports and confusion matrices for both the Decision Tree and Random Forest models.

Interpretation: The same consistent evaluation process is applied to each new model, so results can be compared fairly against both each other and the earlier Logistic Regression results.

15.4 Model Comparison

```
comparison = pd.DataFrame({"Metric": [...], "Decision Tree": [...], "Random Forest": [...]}).set_index("Metric").plot(kind="bar")
```

Purpose: Compare Accuracy, Precision, Recall, and F1-Score between the Decision Tree and Random Forest side by side.

Interpretation: This chart is the deciding evidence for which algorithm to move forward with — typically, Random Forest edges out a single Decision Tree because it's less prone to memorizing quirks of the training data.

16. Feature Importance and Saving the Model

16.1 Ranking Feature Importance

```
importance = pd.DataFrame({"Feature": X_train.columns, "Importance": random_forest.feature_importances_})
importance = importance.sort_values(by="Importance", ascending=False)
sns.barplot(data=importance, x="Importance", y="Feature")
```

Purpose: Extract and rank how much each input feature contributed to the Random Forest's decisions.

Interpretation: This turns the "black box" model into something explainable: it identifies which factors — credit score, payment history, income, and so on — most strongly drive the model's risk assessments, which is essential for justifying decisions to regulators, auditors, or customers.

16.2 Saving the Trained Model

```
import joblib
joblib.dump(random_forest, "credit_risk_model.pkl")
loaded_model = joblib.load("credit_risk_model.pkl")
new_prediction = loaded_model.predict(new_data)
```

Purpose: Save the trained Random Forest model to disk (as a .pkl file) using Joblib, then reload it and confirm it can still make predictions.

Interpretation: This packages the finished model into a reusable file — so it can be plugged into a real application (like a loan approval system) without needing to retrain it every time, and the reload test confirms the saved file actually works.

☐ Risk Status Analysis

☐ Credit Score Analysis

☐ Count Plots

☐ Pair Plots

☐ Scatter Plots

☐ Feature Engineering

- 📄 Label Encoding
- 📄 Train-Test Split
- 📄 KNN Imputer
- 📄 Logistic Regression
- 📄 Confusion Matrix
- 📄 Accuracy, Precision, Recall, F1-Score
- 📄 Type-I & Type-II Errors
- 📄 SMOTE
- 📄 Decision Tree
- 📄 Random Forest
- 📄 Feature Importance
- 📄 Model Saving

17. Conclusion

This project built a machine learning-based **Financial Risk Classification System** to predict whether a customer is **Low Risk** or **High Risk**, based on financial and credit-related information across 10 input features and 1 target variable.

The workflow followed a standard, defensible machine learning process: loading and cleaning the data, filling missing values with the **KNN Imputer**, exploring the data visually, removing outliers, splitting into training and testing sets (80/20), and correcting class imbalance with **SMOTE** before training.

Three models were trained and compared — **Logistic Regression**, **Decision Tree**, and **Random Forest** — using Accuracy, Precision, Recall, F1-Score, and the Confusion Matrix. The **Random Forest** model performed best overall and was selected as the final model, with feature importance analysis used to explain which factors matter most to its predictions.

The final model was saved for reuse, so it can be deployed to score new customers without retraining. In business terms, this system gives a financial institution a faster, more consistent, and more explainable way to flag potentially risky customers — supporting better lending decisions while keeping a close eye on the more costly of the two error types: missing a genuinely high-risk customer (a **Type-II Error**).

Conclusion

This project focused on building a Machine Learning-based Financial Risk Classification System to predict whether a customer falls into the **Low Risk** or **High Risk** category based on their financial and credit-related information. The project followed a structured machine learning workflow, including data loading, data preprocessing, handling missing values using the **KNN Imputer**, and performing **Exploratory Data Analysis (EDA)** to understand the dataset and identify important trends. The dataset contained **10 input features** and **1 target variable**, which were used to train the prediction models.

After preprocessing, feature scaling was applied to improve model performance, and the dataset was divided into **80% training data** and **20% testing data**. Since the dataset contained an imbalanced class distribution, the **Synthetic Minority Over-sampling Technique (SMOTE)** was applied to balance the training dataset, allowing the models to learn effectively from both risk classes and reducing prediction bias.

Three machine learning algorithms—**Logistic Regression**, **Decision Tree**, and **Random Forest**—were trained and evaluated. Their performance was measured using **Accuracy, Precision, Recall, F1-Score, Confusion Matrix**, and **Classification Report**. Among all the models, the **Random Forest** classifier achieved the best performance with an **accuracy of approximately 94%** (replace with your actual accuracy if different), along with high precision, recall, and F1-score. Feature importance analysis also identified the most influential financial attributes contributing to customer risk prediction, helping to improve the interpretability of the model.

The final Random Forest model was successfully saved using the **Joblib library**, enabling it to be reused for future predictions without retraining. Overall, this project demonstrates that machine learning can significantly improve financial risk assessment by accurately identifying high-risk customers, reducing potential financial losses, supporting faster and more informed decision-making, and enhancing the overall credit risk evaluation process. The developed system provides a reliable and scalable solution that can assist financial institutions in making data-driven lending decisions.